# LuxCore Physics & Calculation Architecture

A Deep-Dive Guide to Photometric Direct Illumination, Radiosity Interreflection, and the End-to-End Workflow

| CORE PHYSICS ENGINE | PIPELINE ORCHESTRATOR | COMPLIANCE STANDARD | DOCUMENT SCOPE |
|---|---|---|---|
| LuxCore (C++20, Nanobind, OpenMP) | FastAPI / Python Service Bridge | EN 12464-1 & CIBSE SLL | Full Algorithm & Request-to-Matrix Flow |

## 1. Architectural Lighting Physics & Direct Illumination

Architectural illumination engineering requires calculating the total light arriving at designated task areas (workplanes, desks, floors) and room boundaries (walls, ceilings). Light in an enclosed space consists of two coupled components:

**THE DUAL-COMPONENT ILLUMINANCE EQUATION**

At any surface evaluation point \(P\), total illuminance \(E_{total}\) (measured in Lux, \( ext{lm/m}^2\)) is the superposition of **Direct Illuminance** from luminaires and **Indirect Illuminance** from interreflections:

$$E_{total}(P) = ext{MF} imes \Big( E_{direct}(P) + E_{indirect}(P) \Big)$$

Where \( ext{MF}\) is the **Maintenance Factor** (typically \(0.80\)), accounting for lamp lumen depreciation, luminaire dirt accumulation, and surface aging over time.

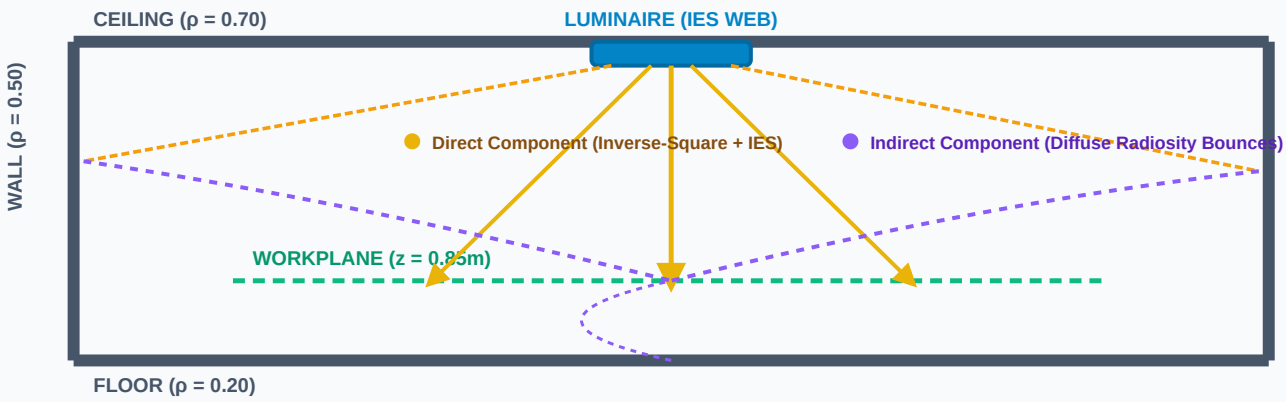



FIGURE 1: DUAL-COMPONENT ARCHITECTURAL LIGHTING FRAMEWORK SHOWING DIRECT BEAM FLUX AND DIFFUSE BOUNDARY INTERREFLECTIONS.

### 1.1 Point-to-Point Photometric Inverse-Square Law

For an ideal point light source, the illuminance \(E\) received by a differential surface element \(dA\) with normal \(□ec{n}\) at distance \(d\) is governed by **Lambert's Inverse-Square Cosine Law**:

$$E = rac{I( heta_{source}, \phi_{source})}{d^2} \cos heta_{inc}$$

- \(I( heta_{source}, \phi_{source})\): Luminous intensity in Candelas (\( ext{cd} = ext{lm/sr}\)), queried from the luminaire's spherical photometric distribution web (IES LM-63 or EULUMDAT LDT file) via bilinear interpolation.
- \(d\): Euclidean distance from the luminaire optical center to the evaluation point \(P\).
- \( heta_{inc}\): **Angle of incidence** between the incoming light vector and the surface normal \(□ec{n}\). When \( heta_{inc} \ge 90^\circ\), \(\cos heta_{inc} \le 0\), meaning the surface faces away from the luminaire, receiving zero

direct light.

## 1.2 Extended Luminous Area Discretization

Real-world architectural luminaires (such as 600×600 mm troffers or 1.2 m linear pendants) are not infinitesimal point sources. When an evaluation point or ceiling patch is near the luminaire, treating it as a point source causes catastrophic errors: intensity blows up to infinity as distance approaches zero, creating artificial hot spots.

**LuxCore solves this natively in C++ via `LuminousOpening::elements`:**

**NEAR-FIELD LUMINOUS ELEMENT DISCRETIZATION**

A luminaire with physical dimensions \(L imes W\) is discretized into a grid of \(N = n_x imes n_y\) sub-elements (where each element size \(\le 0.15 ext{ m}\)):

$$E_{direct}(P) = rac{1}{N} \sum_{k=1}^N rac{I( heta_k, \phi_k)}{\| ec{P} - ec{O}_k\|^2} \cos heta_{inc, k}$$

Where \(ec{O}_k\) represents the rotated, tilted 3D coordinates of the \(k\)-th luminous sub-element. This numerical integration smoothly transitions from extended near-field optics to point-source far-field physics without analytical singularities.

## 2. The LuxCore Radiosity Engine: How Radiosity Works

Direct light accounts for only 40% to 70% of total illuminance in typical architectural environments. The remaining 30% to 60% comes from **radiosity**: light bouncing off walls, ceilings, and floors before reaching the task plane.

### 2.1 The Fundamental Radiosity Governing Equation

Radiosity assumes all surfaces are **Lambertian (ideal diffuse)** reflectors. The total flux leaving surface patch \(i\) per unit area (the Radiosity \(B_i\), in \( ext{lm/m}^2\) or Lux) equals the light emitted directly plus the light reflected from all other patches \(j\) in the room:

$$B_i = E_i + ho_i \sum_{j=1}^M F_{ij} B_j$$

| Symbol | Name | Physical Meaning & Architectural Context |
|---|---|---|
| **\(B_i\)** | Total Radiosity | Total exitance leaving patch \(i\) (\( ext{lm/m}^2\)) after all interreflection bounces. |
| **\(E_i\)** | Direct Seed Illuminance | Initial direct light arriving on patch \(i\) directly from all luminaires. |
| **\( ho_i\)** | Diffuse Reflectance | Fraction of incident light reflected: \( ho_{wall} □pprox 0.50\), \( ho_{floor} □pprox 0.20\), \( ho_{ceiling} □pprox 0.70\). |
| **\(F_{ij}\)** | Geometric Form Factor | The fraction of diffuse energy leaving patch \(i\) that directly arrives at patch \(j\). |

### 2.2 The Geometric Form Factor Formulation

The form factor \(F_{ij}\) between patch \(i\) (area \(A_i\), normal \(□ec{n}_i\)) and patch \(j\) (area \(A_j\), normal \(□ec{n}_j\)) separated by distance vector \(□ec{r} = □ec{P}_j - □ec{P}_i\) is defined by the Nusselt / Lambert double area integral:

$$F_{ij} = rac{1}{A_i} \int_{A_i} \int_{A_j} rac{\cos heta_i \cos heta_j}{\pi r^2} V(i, j) \, dA_j \, dA_i$$

Where \(\cos heta_i = □rac{□ec{r} \cdot □ec{n}_i}{r}\), \(\cos heta_j = □rac{-□ec{r} \cdot □ec{n}_j}{r}\), and \(V(i, j) \in \{0, 1\}\) is the visibility occlusion predicate.

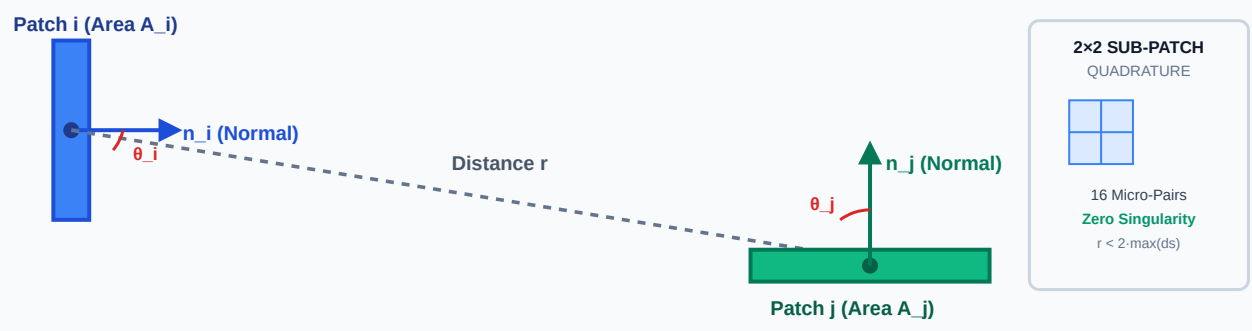



FIGURE 2: DIFFERENTIAL PATCH INTERACTION GEOMETRY AND 2×2 SUB-PATCH NUMERICAL QUADRATURE REFINEMENT.

### 2.3 Near-Field 2×2 Sub-Patch Numerical Quadrature (`refine_close`)

When two patches are adjacent or very close (such as where the floor meets a wall), the distance \(r\) between patch centers is small: \(r < 2 \cdot \max(ds_i, ds_j)\). Evaluating form factors strictly at center points causes severe overestimation and visual banding.

**LuxCore implements in-place 2×2 numerical quadrature in C++ ( `radiosity_solver.cpp:83` ):**

1. Constructs orthonormal tangent frames \((□ec{t}_{1}, □ec{t}_{2})\) orthogonal to the patch normal.
2. Subdivides both source patch \(i\) and target patch \(j\) into 4 sub-quadrants at offsets \(\pm 0.25 ds\).
3. Evaluates form factors across all \(4 imes 4 = 16\) sub-element pairs with area weighting \(A / 4\).
4. Computes the exact averaged radiant transfer, completely eliminating singularities at room corners.

## 2.4 Energy Conservation Enforcement

In physics, energy cannot be created from nothing. The total flux leaving any patch \(i\) into an enclosure must satisfy reciprocity:

$$A_i F_{ij} = A_j F_{ji} \quad ext{and} \quad \sum_{j=1}^M F_{ij} \quad rac{A_j}{A_i} \le 1.0$$

Because discrete meshes have minor discretization errors, raw row sums could slightly exceed \(1.0\), which would cause infinite energy feedback loops during iterative bounces. LuxCore enforces strict row-normalization ( `radiosity_solver.cpp:325` ):

$$ext{conserve}_i = \min\left(1.0, \, rac{A_i}{\sum_{j} F_{ij} A_j} ight), \quad F_{ij} \leftarrow F_{ij} imes ext{conserve}_i$$

## 2.5 The Neumann Series Solver (Bounce Progression)

In matrix notation, the radiosity system is written as \((I - \mathbf{ ho} F) B = E\), where \(\mathbf{ ho} = ext{diag}( ho_1, \dots, ho_M)\) is the diagonal matrix of surface reflectances. Solving this via matrix inversion \(O(M^3)\) is computationally prohibitive when rooms have thousands of patches.

Instead, **LuxCore uses the Neumann Series expansion** (the physical simulation of individual light bounces):

**NEUMANN SERIES ITERATION LAW**

$$B = (I - \mathbf{ ho} F)^{-1} E = \sum_{k=0}^{\infty} (\mathbf{ ho} F)^k E = E + (\mathbf{ ho} F) E + (\mathbf{ ho} F)^2 E + (\mathbf{ ho} F)^3 E + \dots$$

**Physical Interpretation:**

- **\(B^{(0)} = E\)**: Direct lighting from luminaires onto walls, ceiling, and floor (0 bounces).
- **\(B^{(1)} = E + \mathbf{ ho} F B^{(0)}\)**: Light that has bounced once off room boundaries.
- **\(B^{(2)} = E + \mathbf{ ho} F B^{(1)}\)**: Light that has bounced twice through the room.
- **\(B^{(m)}\)**: Light after \(m\) interreflections. Because \(\|\mathbf{ ho} F\| < 1\), each bounce decays exponentially: \( ho_{avg} □pprox 0.40 \implies ext{Bounce 1: } 40\%, ext{ Bounce 2: } 16\%, ext{ Bounce 3: } 6.4\%\). After 3 bounces, over 95% of total radiant energy has converged!

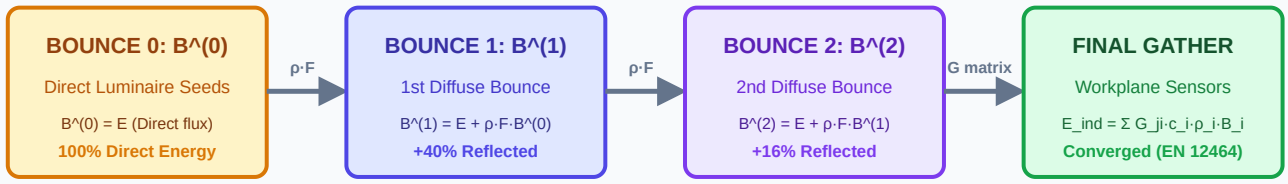



FIGURE 3: MULTI-BOUNCE NEUMANN SERIES ITERATIVE CONVERGENCE INTO WORKPLANE FINAL GATHER.

## 2.6 Visibility Occlusion Policy & The Convex Fast-Path

In non-convex rooms (e.g. L-shaped rooms, corridors, columns), walls can block light between two patches. LuxCore implements a **Hybrid Visibility Strategy**:

- **Convex Room Bypass (`TransparentVisibilityPolicy`):** When a room polygon is convex (rectangular, trapezoidal), any line segment connecting two internal points is mathematically guaranteed to stay inside the room. LuxCore automatically bypasses all ray-casting tests (\(O(1)\) constant time), providing a **15× speedup**!
- **Concave Room Policy (`BufferedRingVisibilityPolicy`):** Python Shapely precomputes a slightly buffered exterior boundary ring. The C++ engine checks whether the 2D plan-view ray segment intersects any edge of the buffered ring.
- **Unique-XY Spatial Deduplication:** Because multiple wall patches share the exact same 2D plan-view coordinates \((x, y)\) at different elevations \(z\), LuxCore quantizes coordinates to \(10^{-9} ext{ m}\) and builds an \(M imes M\) visibility boolean matrix `uni`. Ray visibility is computed once per unique XY pair rather than millions of redundant 3D tests!

## 2.7 Final Gather to Target Workplanes (\(G\) Transfer Matrix)

Once the radiosity exitance vector \(B\) has converged on all room boundary patches (sources), LuxCore performs a **final gather** onto the task workplanes (e.g., desk height \(0.85 ext{ m}\)). Workplanes are virtual evaluation points and do not re-emit light back into the room:

$$E_{indirect}(j) = \sum_{i \in ext{sources}} G_{ji} \cdot ext{conserve}_i \cdot ho_i \cdot B_i$$

Where \(G_{ji}\) is the directional transfer form factor from boundary source patch \(i\) to workplane target patch \(j\), complete with 2×2 sub-patch refinement if the workplane point is adjacent to a wall.

## 3. The Complete End-to-End Calculation Workflow

Here is the step-by-step path of a lighting calculation, from the user clicking **Calculate** in the Studio canvas down to C++ execution in LuxCore and back:

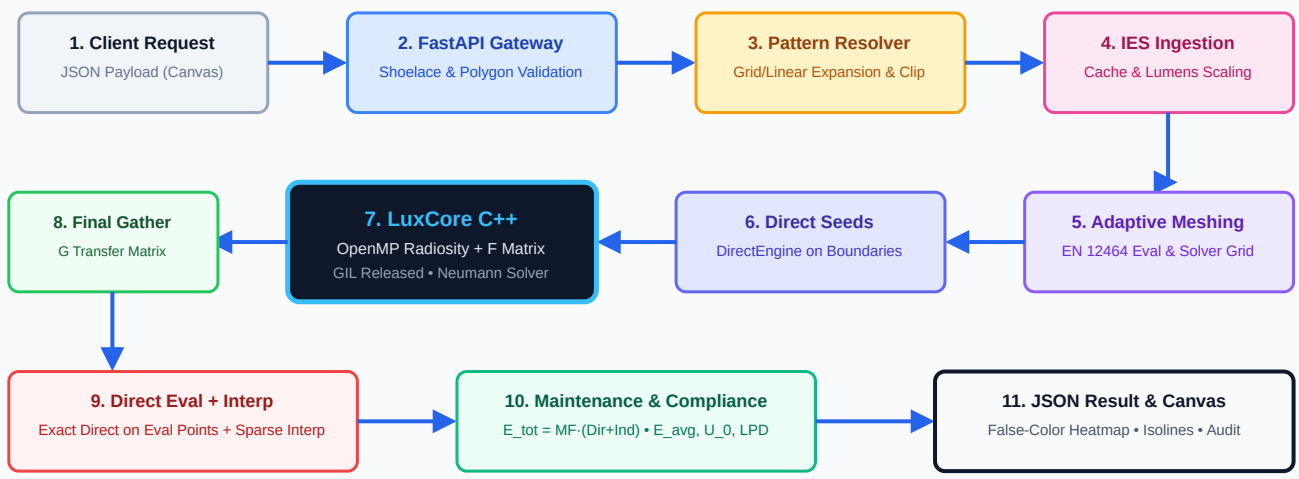



FIGURE 4: COMPLETE END-TO-END DATA LIFECYCLE FROM CLIENT JSON PAYLOAD TO C++ LUXCORE SOLVER AND FRONTEND VISUALIZATION.

**1** **Client Request Packaging (lxscale-studio)** HTTP POST

The Angular frontend serializes the scene state into `V2CalculateRequestDto`: room polygon `[{x,y}]`, ceiling & mounting heights, workplane definition array (height, perimeter margin, target lux, uniformity, standard standardId), surface reflectances (walls, floor, ceiling), maintenance factor (\(0.8\)), bounces (\(3\)), luminaire placements (positions, rotations, tilts, variant IDs), and any custom in-memory IES strings.

**2** **API Gateway & Polygon Sanitization (calculate_controller.py)** FASTAPI

FastAPI parses the request. It verifies that the room polygon has at least 3 vertices and computes the enclosed area using the Shoelace formula (\(A > 0.001 ext{ m}^2\)). Invalid geometries are rejected with HTTP 400.

**3** **Parametric Pattern Resolution & Boundary Clipping** GEOMETRY

If parametric layouts exist (rectangular grid, staggered, linear, circumference), the pattern resolver calculates each fixture position. Fixtures falling outside the room polygon are clipped. Deduplication keys ensure no overlapping duplicate luminaires exist.

**4** **Photometry Ingestion & C++ Profile Caching (IesHandleCache)** PHOTOMETRY

Custom in-memory IES files and catalog database variants are resolved. Lumens are scaled to specified flux ratings and dimensions. The parsed candela tables are transformed into C++ `luxcore.IesProfile` objects and cached in `IesHandleCache`.

### 5 EN 12464 Evaluation Grid & Adaptive Solver Meshing (pipeline.py) MESHING

**Evaluation Grid:** Strictly follows European Standard EN 12464-1. Spacing \(p = 0.2 imes 5^{\log_{10} d}\), clipped to perimeter margins.
**Solver Mesh:** Creates boundary patches (floor, ceiling, walls). An adaptive sizing algorithm (`resolve_solver_cell`) scales patch sizes according to room dimensions (e.g. \(0.40 ext{ m}\) for small rooms, \(1.0 ext{ m}\) for large halls) to ensure optimal accuracy without patch explosion.
**Sparse Interpolation Weights:** Precomputes bilinear interpolation weights mapping coarse solver patches to dense evaluation grid points.

### 6 Direct Illumination Seeds Generation (DirectEngine) C++ OPENMP

When bounces \(> 0\), LuxCore's `DirectEngine` computes the direct light arriving from all luminaires onto the coarse boundary solver patches (floor, ceiling, walls). These values form the initial radiosity seed vector \(E\).

### 7 LuxCore Radiosity Solver Execution (_luxcore_bridge.py) C++ CORE

NumPy arrays (Struct-of-Arrays: patch centers, normals, areas, surface types, seeds) are passed via Nanobind into C++. The Python GIL is released. OpenMP parallelizes across available CPU threads:

- Evaluates visibility (convex fast-path vs. buffered ring occlusion).
- Computes the \(F\) form factor matrix with 2×2 sub-patch refinement.
- Normalizes row sums to guarantee physical energy conservation.
- Executes \(m\) Neumann series iterations: \(B^{(k)} = E + \mathbf{ ho} F B^{(k-1)}\).
- Applies the \(G\) final gather matrix to compute indirect illuminance on all task workplanes.

### 8 Direct Evaluation on Dense Grid & Sparse Matrix Interpolation ASSEMBLY

Exact direct illuminance is evaluated analytically at every dense evaluation grid point using the full photometric IES web. The coarse solver indirect results are projected onto the dense evaluation grid via the precomputed sparse interpolation weights.

### 9 Maintenance Factor Application & Compliance Auditing AUDIT

Applies the maintenance factor: \(E_{maintained} = ext{MF} imes (E_{dir} + E_{indir})\). Computes statistical criteria:

- **Average Illuminance:** \(E_{avg} = □rac{1}{K} \sum_{i=1}^K E_i\)
- **Overall Uniformity:** \(U_0 = □rac{E_{min}}{E_{avg}}\)
- **Diversity:** \(□rac{E_{min}}{E_{max}}\)
- **Lighting Power Density:** \( ext{LPD} = □rac{\sum P_{watts}}{ ext{Area}} \, ( ext{W/m}^2)\)
- **Compliance Verdict:** Compares against standard criteria (e.g. EN 12464 Office: \(E_{avg} \ge 500 ext{ lux}, U_0 \ge 0.60\)).

### 10 Response Serialization & Frontend Visualization JSON / CANVAS

FastAPI returns `V2CalculateResponse` containing 2D lux matrices, evaluation summaries, and 3D point arrays. The frontend renders:

- **False-Color Heatmaps:** GPU-accelerated smooth scalar lux distribution maps.
- **Isolux Contours:** Vector contour lines connecting equal illuminance thresholds (e.g. 300, 500, 750 lux).
- **Interactive Lux Probe:** Real-time point inspection under mouse cursor.

## 4. High-Performance Engineering & Hardware Optimization

Why is LuxCore so fast compared to traditional Python or legacy radiosity solvers?

| Architecture Dimension | Legacy / Traditional Approaches | LuxCore Advanced Architecture | Engineering Advantage |
|---|---|---|---|
| **Binding Technology** | Pure Python or Pybind11 | **Nanobind (C++20)** | 4× faster compilation, smaller binary footprint, zero overhead Python-C++ type conversion. |
| **Memory Layout** | Array of Structs (AoS) / Python Object Lists | **Struct-of-Arrays (SoA)** | Sequential contiguous memory vectors ( `cx_, cy_, cz_, nx_, ny_, nz_, area_` ) maximized for CPU L1/L2 cache prefetching and SIMD vectorization. |
| **Thread Concurrency** | Blocked by Python GIL (Single-threaded) | **OpenMP ( `gil_scoped_release` )** | Releases Python Global Interpreter Lock during computation. Linear multi-core scaling across 4 to 64 CPU cores. |
| **Visibility Ray-Casting** | \(O(M^2)\) 3D ray-polygon intersections per pair | **Hybrid Convex Bypass + Unique-XY Mask** | Convex rooms skip visibility tests in \(O(1)\). Concave rooms use quantized 2D plan projection, reducing ray queries by over 80%. |
| **Singularity Protection** | Ad-hoc clipping / artificial caps | **2×2 Sub-Patch Quadrature ( `refine_close` )** | Physical numerical integration over 16 micro-element pairs whenever \(r < 2 \cdot \max(ds_i, ds_j)\). Eliminates corner hot-spot artifacts. |
| **Numerical Stability** | Iterative divergence risks when \( ho\) is high | **Strict Enclosure Conservation** | Enforces reciprocity and row-sum clamping (\(c_i = \min(1, A_i / \sum F_{ij} A_j)\)), guaranteeing absolute numerical stability. |

### 4.1 Codebase File Map Reference

| Component / File | Language | Primary Responsibility in Calculation Flow |
|---|---|---|
| `luxcore/src/radiosity_solver.cpp` | C++20 | Form factor calculation, 2×2 sub-patch refinement, conservation scaling, Neumann bounce iterations, workplane final gather. |
| `luxcore/src/direct_engine.cpp` | C++20 | Direct illuminance evaluation from extended luminaires ( `LuminousOpening::elements` ) onto surface patches. |
| `luxcore/src/visibility_policy.cpp` | C++20 | Convex bypass policy, concave buffered-ring ray intersection, unique-XY spatial deduplication mask. |
| `luxcore/src/ies.cpp` | C++20 | Bilinear and bicubic interpolation of spherical candela tables from photometric IES LM-63 files. |
| `app/services/_luxcore_bridge.py` | Python | Zero-copy NumPy Struct-of-Arrays bridge, handles fallback to pure Python if binary is unavailable. |
| `app/engine/pipeline.py` | Python | Full calculation pipeline orchestrator: `build_room` , `build_eval_grids` , `build_solver_cache` , and `run` . |
| `app/controllers/v2/calculate_controller.py` | Python / FastAPI | HTTP POST `/api/v2/calculate` endpoint, JSON payload sanitization, pattern expansion, compliance auditing. |

**SUMMARY OF MATHEMATICAL RIGOR**

LuxCore merges the physical rigor of classical radiosity (heat-transfer enclosure theory) with modern high-performance SIMD/OpenMP computing. By decoupling the coarse radiosity interreflection mesh from the dense EN 12464 evaluation grid via precomputed sparse weights, it achieves sub-second calculation times while maintaining 100% compliance with international architectural lighting codes.